

Natural Language Processing

Project 1.4: Sentiment Analysis on Twitter Data

The goal of this project is to apply Natural Language Processing (NLP) methods to a real-world sentiment analysis task. You will investigate how different preprocessing methods, feature representations, and machine learning models affect sentiment classification performance. The primary goal is not to achieve the highest possible classification score. Instead, you should focus on understanding how different methodological decisions influence model behavior and performance. Particular emphasis will be placed on error analysis, critical reflection, and reproducibility.

Organization

- Please read the project description carefully. All the details are presented in this document. If you have questions, please write to roland.roller@dfki.de or post in the public forum (and possibly notify me).
- Students may work individually or in groups of up to two students. In the case of group work, both students should contribute equally. Only one submission per group is required.
- Your solution should contain your a) source code, the b) report and the c) experiment summary sheet.
- **Naming convention:** Zip all files and name the zip-file NLP_project_1_1_[Group Name].zip or NLP_project_1_1_[Surname(s)].zip
- **Submission Deadline:** June 16, 2026 (23:55, GMT+2)
- You can make unlimited re-submissions, until the deadline, but the latest submission will be graded.
- For the report, please use the ACM proceedings template from here (<https://www.acm.org/publications/proceedings-template>). The Office Word and latex templates are provided on the page. Submit the report as PDF.
- **Length report:** Each report should have a length of a maximum of 4 pages.
- Programming:
 - Please use Python 3 for all coding tasks.
 - You are free to use any package/module/library unless a task explicitly forbids it.
 - Code must be documented, readable, and reproducible.

- Avoid unnecessary file fragmentation:
- Prefer one script or one script per task.
- You may submit code as .py files or Jupyter notebooks.

Plagiarism Statement

Your project, including the report and the code, will be checked against other submissions in the class. We trust you all to submit your own work only. Copying someone else's code and report and submitting it with minor changes, will be treated as plagiarism.

Use of Generative AI Tools

The use of generative AI tools for learning, brainstorming, and improving the presentation of results is permitted. However, generative AI tools must not be used to generate implementation code for this project. All submitted code must be written and understood by the students themselves.

Introduction

In this project, you will develop and evaluate different Natural Language Processing methods for sentiment analysis of tweets. The dataset consists of short texts labeled as positive, neutral, or negative sentiment. The main goal of this project is not to maximize classification performance, but to understand how different modeling decisions influence results and error behavior. You are expected to carefully analyze model errors, compare different approaches, and reflect on why certain methods succeed or fail. Across all tasks, emphasis is placed on experimental reasoning, interpretation of results, and reproducibility rather than fine-tuning models for marginal performance gains.

Data

In this project, you receive a zip file containing a CSV file. There are a number of columns in the files including textID, text, selected_text, sentiment (positive, negative, neutral), Time of Tweet, and so on. Please keep text and sentiment as the input and output features and ignore the other columns. Moreover, you can sample a part of the dataset, if it's too big to train a model on your machine. Otherwise, you can use the whole dataset to train your models.

Task 1: Exploratory Data Analysis (15%)

Before building any models, you should perform an exploratory analysis of the dataset to understand its structure and characteristics. This includes examining the distribution of

sentiment classes, the length of the texts, and properties of the vocabulary. You are also encouraged to investigate frequent words or n-grams and to use visualizations such as word clouds or distribution plots where appropriate.

Your report should not only present descriptive statistics and figures but also interpret them. In particular, you should explain what the observed patterns imply for the sentiment classification task and what challenges they may introduce.

Task 2: Text Preprocessing (10%)

In this task, you will apply preprocessing steps to prepare text data for modeling. You are expected to make informed design decisions based on the characteristics of the dataset and the requirements of the task. Typical preprocessing steps may include lowercasing, tokenization, stop-word removal, stemming or lemmatization, punctuation removal, or handling of numbers. However, you are not required to apply all possible methods. In your report, you should briefly explain which preprocessing choices you made and why you considered them useful. You should also reflect on which steps you decided not to apply and why.

Task 3: Sentiment Classification (35%)

In this task, you will build classical machine learning models for sentiment classification. You should begin by splitting the dataset into training and test sets using an 80/20 split. If necessary, you may apply techniques to handle class imbalance, such as resampling or class weighting.

You will first train a Naive Bayes classifier and evaluate it on the test set. You should experiment with different text representations, such as bag-of-words models or TF-IDF representations, and observe how these choices affect performance.

Next, you will train a feed-forward neural network for the same task using appropriate text representations. Again, you should investigate how different feature representations and preprocessing decisions influence results.

For both models, you should report standard evaluation metrics including accuracy, precision, recall, and macro F1 score, and include a confusion matrix.

A central component of this task is error analysis. You are expected to carefully examine incorrectly classified examples and discuss possible reasons for the observed errors. This may include ambiguity in tweets, insufficient context, preprocessing effects, or class imbalance.

Finally, you should compare the Naive Bayes model with the neural network model and discuss the differences in performance and behavior. In addition, you will repeat the classification task in a binary setting by removing the neutral class and compare results between the binary and multiclass setups, focusing on differences in difficulty and error patterns.

Task 4: Textual similarity (15%)

In this task, you will analyze semantic similarity between tweets. You should select 15 examples of negative sentiment from the dataset and represent each sentence by averaging its word vectors to obtain a simple sentence-level representation. Based on these representations, you will compute pairwise cosine similarities between sentences. The similarity computation should be implemented manually without using pre-built similarity functions.

In your report, you should present the resulting similarity values and discuss which sentence pairs are most similar and which are least similar, providing an interpretation of these findings.

Task 5: Pre-trained Language Models (25%)

In this task, you will fine-tune one or two pre-trained transformer-based language models on the provided dataset and evaluate their performance in both the binary and multiclass sentiment classification settings. You may choose from a range of widely used lightweight and efficient pre-trained models, such as DistilBERT, MiniLM, or other BERT based models.

When fine-tuning the models, carefully consider important training parameters such as learning rate, batch size, number of epochs, and regularization strategies. The goal is to adapt the model to the dataset while preserving useful knowledge from pre-training and avoiding overfitting.

In your report, clearly describe the selected models, the training setup, and the experimental procedure. In addition, compare the results of the fine-tuned transformer models with the classical machine learning approaches from the previous tasks. Discuss differences in performance, typical error cases, and the computational cost of training and inference, with particular emphasis on model behavior rather than only final performance scores.